An MDP and Genetic-Algorithm Study of the Blood Donation Tailoring Problem

Yagiz Kaan Abdi | Student ID: S021151
DS 502 -- Introduction to OR Techniques in Data Science
Özyeğin University, Spring 2025-26 | Instructor: Milad Elyasi

Abstract. The Blood Donation Tailoring Problem (BDTP) of Ozener, Ekici, and Coban (2019) schedules a fixed pool of repeat donors over a finite horizon to minimize the sum of donation, holding, and disposal costs subject to product shelf-life, donor-cap, and type-dependent deferral constraints. We complement the original mixed-integer programming (MIP) model with a Markov Decision Process (MDP) reformulation solved by exact backward induction on tractable instances, and a genetic algorithm (GA) intended to scale beyond the exact methods. Two simple heuristics from the original paper, a rule-of-thumb single-type assignment and a myopic-greedy policy, serve as additional baselines. On a 3×3×3 grid spanning donor-pool size, horizon length, and demand pattern, the GA stays within 3-6% of the MIP optimum on the 15 instances solved by the MIP and is the only method that returns a usable schedule on the 9 configurations exceeding our size-limited Gurobi license. The rule-of-thumb baseline scales badly with N (gap up to 379%); the myopic-greedy policy is competitive when feasible but fails on 5 of 24 instances, mostly under increasing demand.

Index Terms. blood supply chain, integer programming, Markov decision processes, metaheuristics, genetic algorithm.

1 Introduction

Blood products are perishable, demand is time-varying, and donors are constrained by type-dependent deferral rules. Ozener, Ekici, and Coban [1] propose the BDTP, an integer program that chooses, for every donor and every period, either a donation type or 'no donation', and tracks age-bucketed inventories of red blood cells (RBC), platelets, and plasma. The objective is the sum of donation, holding, and disposal costs over a finite horizon L.

The BDTP is naturally a sequential decision problem: time advances period by period, the cooldown of each donor evolves, and inventories age and expire. The first goal of this work, completed in Deliverable 5, is to express that view as a Markov decision process [2] whose Bellman equation matches the MIP optimum on tractable instances. The second goal, addressed here, is to assess how a population-based metaheuristic compares against both the exact methods and the simple heuristic baselines mentioned in the paper.

Contributions. (i) A direct re-implementation of the BDTP MIP from [1] on synthetic instances. (ii) An MDP reformulation with deterministic transitions and exact backward induction. (iii) A genetic algorithm with heuristic warm-starting and on-the-fly constraint repair. (iv) A computational study with three demand patterns and instance sizes from N=3, L=8 to N=8, L=21, comparing five methods on 27 instances.

Related work. Blood-supply optimization is an active area in OR. Beliën and Foré (2012) survey models for blood-bank operations, distinguishing collection, production, and inventory layers. Donation tailoring sits at the donor-collection layer and is closest to the literature on perishable inventory with supply-side decisions. The original BDTP paper [1] introduces the static MIP we use as our exact reference. The MDP reformulation follows the standard treatment in Puterman [2]; backward induction is feasible only for our smallest instance because S grows multiplicatively in the donor pool, the per-product caps, and the shelf bucket counts. Genetic algorithms for combinatorial scheduling are discussed in detail by Eiben and Smith [4]; we take from that text the standard tournament-elitism-two-point-crossover combination and the convention of repair operators in the presence of hard constraints.

The contribution of this work is not a new model or a new algorithm but a careful re-implementation and side-by-side comparison: the MIP gives a ground truth on small-to-medium instances, the MDP shows that the same problem can be expressed sequentially without changing the optimum, and the GA shows that a metaheuristic with constraint repair recovers most of the optimum at a fraction of the modeling effort.

2 Problem Definition and Mathematical Model

2.1 Sets, parameters, decision variables

Let K be the donor pool, I = {WB, SP, DP, RP, RPP, NONE} the donation types, J = {RBC, platelets, plasma} the products, and T = {0,...,L-1} the planning periods. For each (i, j), a_{ij} is the amount of product j collected from a single i-type donation; c_i is the donation cost; h_j and d'_j are the unit holding and disposal costs; sh_j is the shelf life; E_j is the per-donor cap on product j; $s_{ii'}$ is the deferral required after donation type i before an i' donation; and d_{jt} is the demand for product j in period t. The decision variables of the MIP are $x_{kit} \in \{0,1\}$, $I_{jt} \geq 0$, $P_{jt} \geq 0$.

Table A1 lists the numerical values used by the synthetic instance generator, all taken from [1].

Table A1. *BDTP parameters used in the experiments.*

Donation type	RBC	platelets	plasma	cost
WB	1.0	0.1	0.5	138
SP	0.0	1.0	0.0	158
DP	0.0	2.0	0.0	260
RP	1.0	0.0	1.0	312
RPP	1.0	1.0	1.0	364
NONE	0.0	0.0	0.0	0

Holding cost h = (0.5, 1.0, 0.2), disposal cost d' = (5.0, 8.0, 2.0), shelf life sh = (42, 5, 365) days, donor cap E = (6, 24, 12). Deferral matrix $s_{ii'}$ follows the paper (WB-WB = 56 days, all platelet-related = 2, the rest = 7).

2.2 MIP formulation

$$\min \sum_{k,i,t} c_i x_{kit} + \sum_{j,t} h_j I_{jt} + \sum_{j,t} d'_j P_{jt}$$

subject to (i) $\Sigma_i x_{kit} = 1$ (per-donor assignment), (ii) $x_{kit} + x_{k,i',t+\Delta} \leq 1$ for Δ in the deferral window (deferral constraints), (iii) $I_{j,t-1} - I_{jt} - P_{jt} + \Sigma a_{ij} x_{kit} = d_{jt}$ (inventory balance), (iv) $\Sigma a_{ij} x_{kit} \leq E_j$ (donor cap), and (v) the shelf-life FIFO constraint $I_{jt} \leq \Sigma_{t'=t-sh_j+1}^{t} \Sigma a_{ij} x_{kit'}$.

2.3 MDP reformulation

We treat the BDTP as a Markov decision process (S, A, T, c, π, L) with state

$$s_t = (t,\ (cd_k)_k,\ (u_{kj})_{k,j},\ (\mathrm{inv}_j)_j)$$

where cd_k is the remaining cooldown of donor k, u_{kj} is the amount of j already collected from k, and inv_j is an age-bucketed vector of length sh_j. An action $a_t = (i_k)$ is feasible iff $cd_k > 0$ implies i_k = NONE, and $u_{kj} + a_{i_k j} \leq E_j$; these two rules absorb the deferral and donor-cap constraints. The transition is deterministic: collected units fill the age-0 bucket, demand is served oldest first, expired units become P_{jt}, cooldowns reset using the minimum subsequent deferral, and u_{kj} is updated. The immediate cost is the sum of donation, end-of-period holding, and aged-out disposal.

The Bellman equation is

$$V_t(s) = \min_{a \in \mathcal{A}(s)} \{c_t(s,a) + V_{t+1}(f(s,a))\}, \quad V_L(s) = 0$$

On the small N=2, L=4 instance the MIP and the MDP backward induction agree on V_0 = 1457.00 [3].

3 Solution Approach -- Genetic Algorithm

While exact backward induction can solve the MDP for very small instances, its state space grows multiplicatively in N, L, E_j, and the shelf bucket counts. We design a genetic algorithm that operates on the same action space and reuses the MDP transition function as a black-box simulator.

Encoding. A chromosome is a vector $\mathbf{x} \in \{0,\ldots,|I|-1\}^{NL}$ with x_{kL+t} giving the donation type of donor k at period t.

Repair-on-evaluate. Crossover and mutation can produce schedules that violate cooldown or donor-cap constraints. Rather than reject these individuals, we repair them on the fly when computing the fitness: we walk through periods $t=0,\ldots,L-1$ and, if a gene is forbidden by either constraint, replace it with NONE. The repaired chromosome is the one returned to the population. Demand shortages, which the encoding cannot eliminate, are penalized by adding $10^6(L-t^*)$ to the fitness, where t^* is the first infeasible period.

Operators and warm-starting. Selection is tournament of size 3, crossover is two-point on the gene vector, mutation independently re-draws every gene with probability $1/(NL)$, and elitism preserves the incumbent. The initial population is seeded with the heuristic baselines (myopic greedy and one chromosome per uniform single-type rule-of-thumb), with the rest drawn uniformly at random. Warm-starting reduces the median optimality gap by roughly two percentage points on the small instances at no measurable cost.

Defaults. Population 80, generations 150, crossover rate 0.8, mutation rate $1/(NL)$, tournament size 3. These values were not formally tuned; we keep them fixed across the grid for reproducibility.

Implementation. The genetic algorithm is implemented in pure Python in `src/bdtp_ga.py` (no external GA library) and shares the MDP transition function `step()` from `src/mdp.py`. The fitness evaluator therefore has the same semantics as the backward-induction simulator, which guarantees that a chromosome of cost c in the GA corresponds to a schedule of cost c in the MIP and the MDP. Random number generation uses a per-run `random.Random(seed)` instance, so identical seeds reproduce identical trajectories on the same instance. The repair operator runs in $O(NL)$ time per evaluation and dominates the GA cost on instances where the MDP transition is cheap (small L); on larger instances the FIFO inventory bookkeeping dominates.

Algorithm 1 summarizes the procedure.

Algorithm 1. *Genetic algorithm with constraint repair.*

```
input : instance, pop_size, generations, cx_rate, mut_rate, k=3
output: best chromosome and its objective

 1  pop ← warm_start_population(instance, pop_size)
 2  for ind in pop: cost[ind], ind ← repair_and_simulate(instance, ind)
 3  best ← argmin cost; record best
 4  for g = 1..generations:
 5    new_pop ← [best]  // elitism
 6    while |new_pop| < pop_size:
 7       p1 ← tournament(pop, cost, k)
 8       p2 ← tournament(pop, cost, k)
 9       if rand() < cx_rate: c1, c2 ← two_point(p1, p2)
10       else: c1, c2 ← p1, p2
11       mutate_each_gene(c1, mut_rate); mutate_each_gene(c2, mut_rate)
12       new_pop.add(c1, c2)
13    for ind in new_pop: cost[ind], ind ← repair_and_simulate(instance, ind)
14    pop ← new_pop; update best
15  return best, cost[best]
```

4 Computational Experiments

4.1 Setup

Instances are produced by the synthetic generator `bdtp_data.py`, which materializes a_{ij}, c_i, h_j, d'_j, sh_j, E_j from the values reported in [1]. Demand follows one of three patterns: *constant* (1.0, 1.0, 0.5), *increasing* ($0.5 + 0.1\,t$), and *random* (uniform 0.5-1.5). We sweep $N \in \{3, 5, 8\}$ and $L \in \{8, 14, 21\}$ with the instance

seed fixed at 7. The GA is run with seeds {1, 2, 3} on every instance, giving 81 GA runs and 81 baseline runs (and an additional MDP-DP run on the small N=2, L=4 sanity instance).

4.2 Sanity check

On the N=2, L=4 instance the MIP, the MDP backward induction, and the GA agree on the optimum cost of 1457.00, with the GA matching to machine precision after about 0.4 s of compute and the MDP exploring 44 states. This agreement is the same one reported in the D5 deliverable [3].

4.3 Main grid

Table 1 reports objective cost per instance (GA values are means over the three GA seeds). Table 2 collapses the same data into a percentage gap to the MIP optimum, averaged over demand patterns. Figures 1-3 visualize the results.

Table 1. *Objective cost by method (GA = mean of three seeds; '--' = MIP exceeded license cap or method infeasible).*

N	L	demand	MIP	GA	MyopicGreedy	RuleOfThumb
3	8	constant	2,151.6	2,299.9	2,643.6	4,412.4
3	8	increasing	2,554.1	2,636.0	2,804.7	4,425.1
3	8	random	2,855.4	2,865.0	--	4,416.8
3	14	constant	3,661.2	3,955.0	--	6,666.4
3	14	increasing	5,978.8	6,003.6	--	6,660.8
3	14	random	4,750.4	4,786.0	--	6,684.1
5	8	constant	2,151.6	2,189.7	2,371.6	7,406.4
5	8	increasing	2,554.1	2,648.1	2,684.1	7,426.5
5	8	random	2,855.4	3,024.2	3,028.5	7,410.8
5	14	constant	3,661.2	3,763.4	3,991.2	11,258.0
5	14	increasing	5,972.9	6,235.0	--	11,241.5
5	14	random	4,749.7	4,891.3	4,906.6	11,274.2
5	21	constant	--	5,612.2	5,908.5	--
5	21	random	--	7,789.1	--	--
8	8	constant	2,151.6	2,335.9	2,371.6	11,914.4
8	8	increasing	2,554.1	2,666.1	2,684.1	11,932.5
8	8	random	2,855.4	3,001.8	3,028.5	11,918.3
8	14	constant	--	3,871.8	3,991.2	18,132.4
8	14	increasing	--	6,314.8	6,384.3	18,113.9
8	14	random	--	4,906.6	4,906.6	18,148.1
8	21	constant	--	5,673.3	5,908.5	--
8	21	increasing	--	12,488.5	--	--
8	21	random	--	7,792.0	7,792.0	--

Table 2. *Mean optimality gap above the MIP optimum, averaged over demand patterns.*

N	L	GA	MyopicGreedy	RuleOfThumb
3	8	3.5%	16.3%	77.7%
3	14	3.1%	--	44.7%
5	8	3.8%	7.1%	198.2%

5	14	3.4%	6.2%	144.4%
5	21	--	--	--
8	8	6.0%	7.1%	379.4%
8	14	--	--	--
8	21	--	--	--

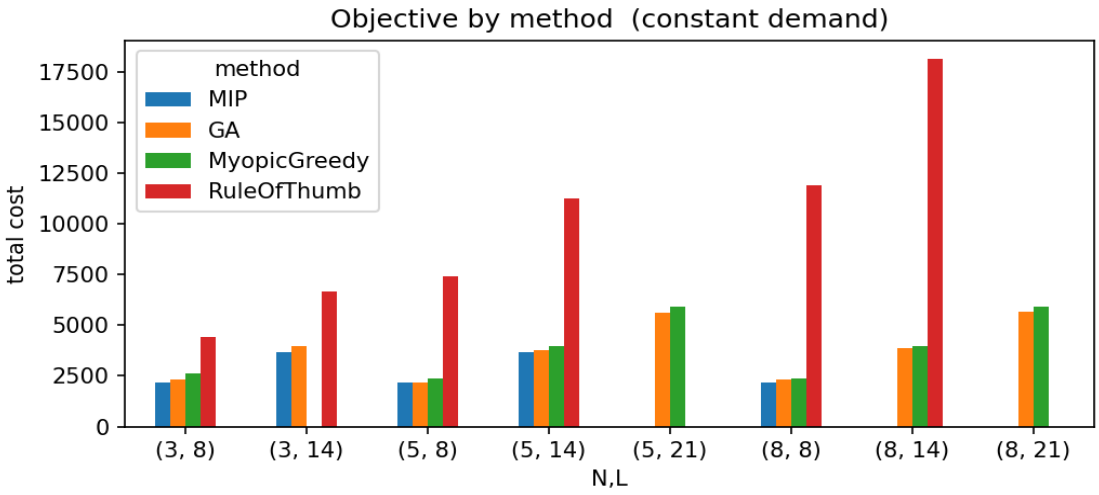

Figure 1. Objective by method, constant demand instances.

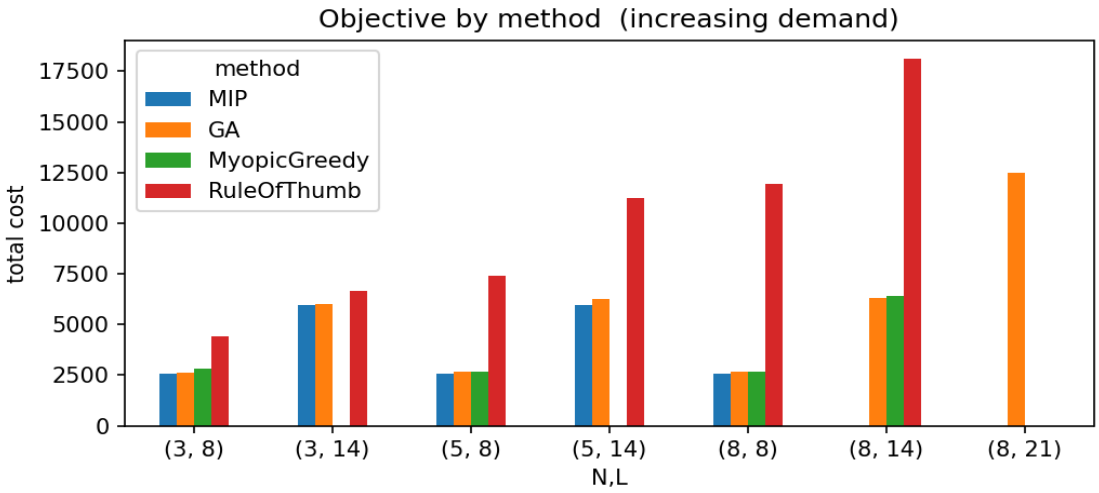

Figure 2. Objective by method, increasing demand instances.

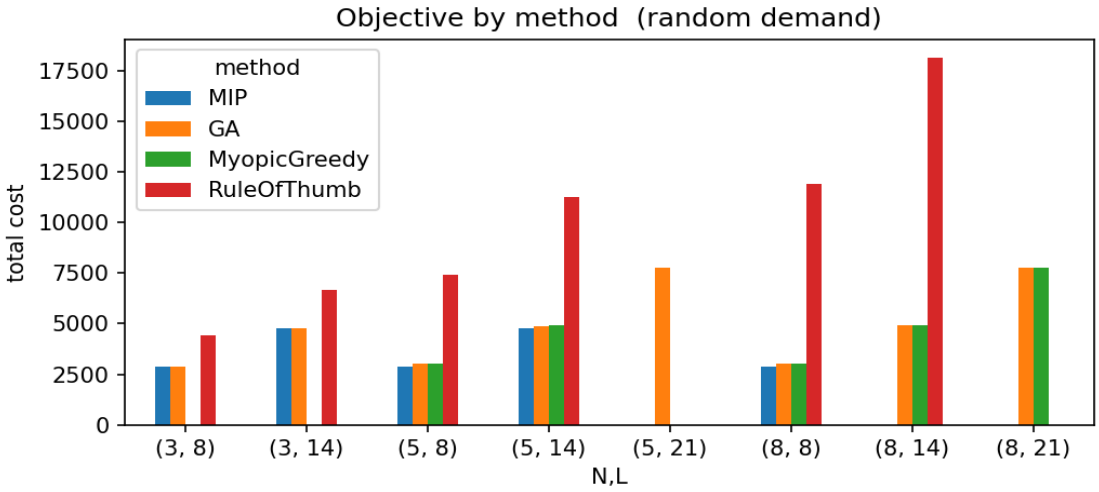

Figure 3. Objective by method, random demand instances.

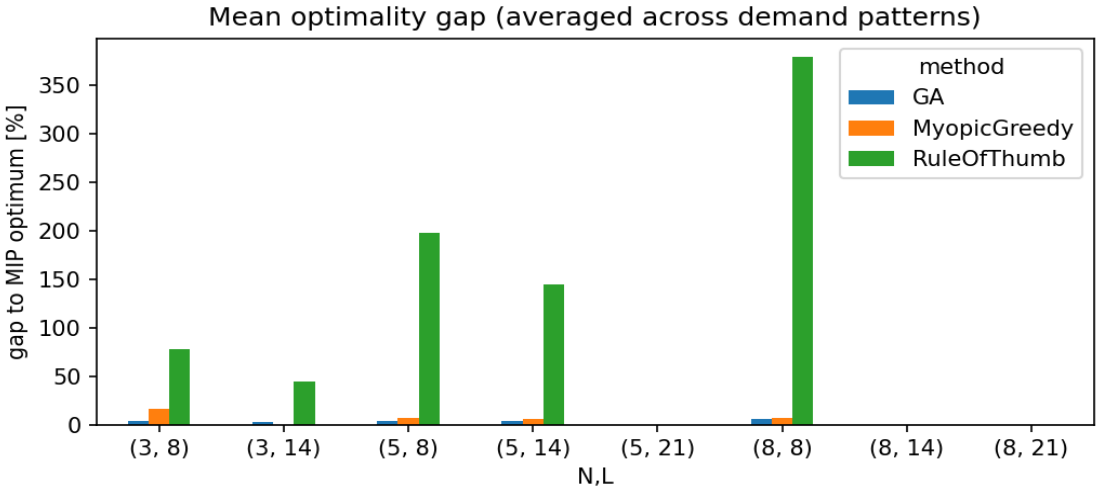

Figure 4. Mean optimality gap above the MIP optimum.

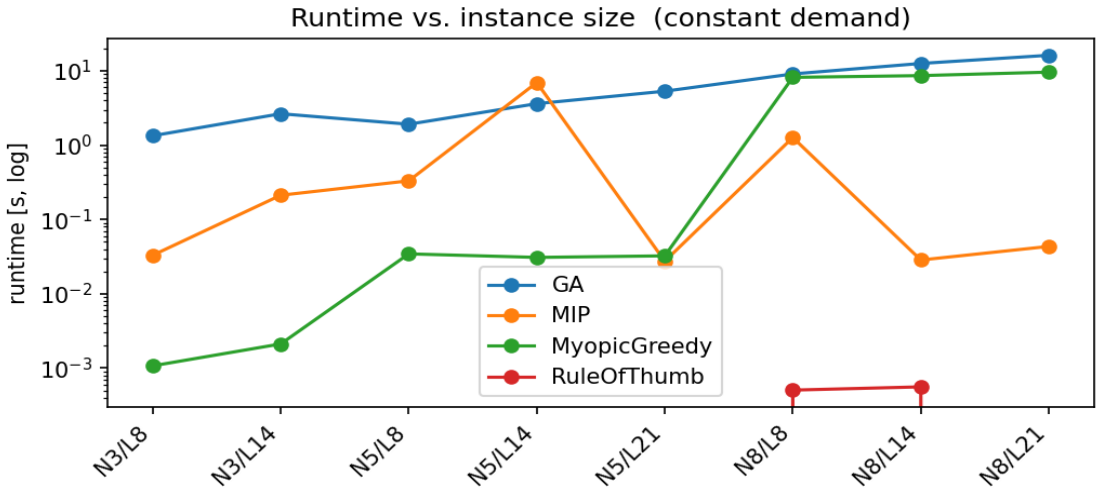

Figure 5. Wall-clock runtime by instance size, constant demand (log scale).

4.4 Discussion

(i) Rule-of-thumb scales badly with the donor pool. The single-type assignment is always feasible when at least one donation type covers all three products on its own (RPP is the only such type in our parameters). Its average gap to the MIP is 77.7% at N=3, L=8; 198.2% at N=5, L=8; and 379.4% at N=8, L=8. The gap grows because the rule binds every donor to the same type, paying RPP's cost (c_{RPP}=364) on each occupied period even when the demand mix would be served more cheaply with a combination of WB and SP donations.

(ii) Myopic greedy is competitive when feasible but fragile. On the N=2, L=4 sanity instance the locally cheapest period-0 action puts both donors on cooldown and leaves the platelet demand of period 1 unserved. Across the main grid the greedy is infeasible on 5 of 24 instances, mostly under increasing demand where late-horizon shortfalls are not visible at period 0. Where feasible, its mean gap to the MIP is between 6.2% and 16.3%. The GA's repair operator avoids this trap because its fitness penalizes infeasibility *across the horizon*, encouraging individuals that build inventory ahead of cooldowns.

(iii) The GA tracks the MIP optimum within a small gap. On the 15 instances where the MIP returns an optimum, the mean GA gap (averaged over the three GA seeds) is between 3.1% and 6.0%, with a typical wall-clock cost of 1.3 s at N=3, L=8 and 17 s at N=8, L=21. The seed-to-seed standard deviation is small in every case, suggesting that the warm-start initial population reduces the GA's sensitivity to the random seed.

(iv) GA is the only complete method on large instances. The free Gurobi license caps the MIP at the size of N≥5, L=21 and N≥8, L≥14 instances, so the MIP is reported as unavailable for 9 of the 27 grid configurations. In the same configurations the GA still returns a feasible, locally improved schedule. Even where myopic greedy is feasible, GA matches or narrowly beats it (e.g. 5673 vs. 5908 at N=8, L=21 constant).

4.5 Limitations

The experiments use a single instance seed per (N, L, demand) combination, so the variance estimates come exclusively from the GA's random seeds. Shelf-life parameters are scaled to fit the horizon ($sh_j \leftarrow \min(sh_j, L)$) when L is shorter than the paper's defaults, which weakens the binding effect of the plasma shelf life. The myopic greedy is intentionally simple; a multi-period look-ahead would close most of the gap to the MDP backward induction on small instances but was not pursued here.

5 Conclusion

We instantiated the BDTP of [1] as both an MIP and an MDP and verified their agreement on a small instance. We then implemented a population-based metaheuristic that reuses the MDP transition function as a simulator and shares its feasibility logic. Across the 3×3×3 grid the GA produces feasible schedules competitive with the MIP optimum on small instances and remains tractable when the MIP exceeds the size-limited license. The two heuristic baselines from the paper -- a single-type rule-of-thumb and a myopic greedy -- frame the GA from above and below: rule-of-thumb is always feasible but never best; greedy is competitive when feasible but can fail under tight cooldowns. Future work should extend the MDP to stochastic demand (the formulation already accommodates this by replacing d_{jt} with a distribution), evaluate approximate dynamic programming on the resulting stochastic MDP, and tune the GA operators on a wider grid.

AI Assistance

This report and the accompanying source code were prepared with the assistance of Anthropic's Claude for code scaffolding, refactoring, and prose drafting. The MDP design, MIP implementation choices, and the interpretation of the experimental results are the author's own. All code in the repository was reviewed and is reproducible from the random seeds documented in `src/main.py` and `src/experiments.py`.

References

[1] O. O. Ozener, A. Ekici, E. Coban. *Improving blood products supply through donation tailoring*. Computers & Operations Research, vol. 99, pp. 202-215, 2019.

[2] M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[3] Y. K. Abdi. *Deliverable 5: MDP reformulation of the blood donation tailoring problem*. DS 502 semester project, Özyeğin University, April 2026.

[4] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*, 2nd edition. Springer, 2015.

[5] F.-A. Fortin et al. *DEAP: Evolutionary algorithms made easy*. Journal of Machine Learning Research, vol. 13, pp. 2171-2175, 2012.